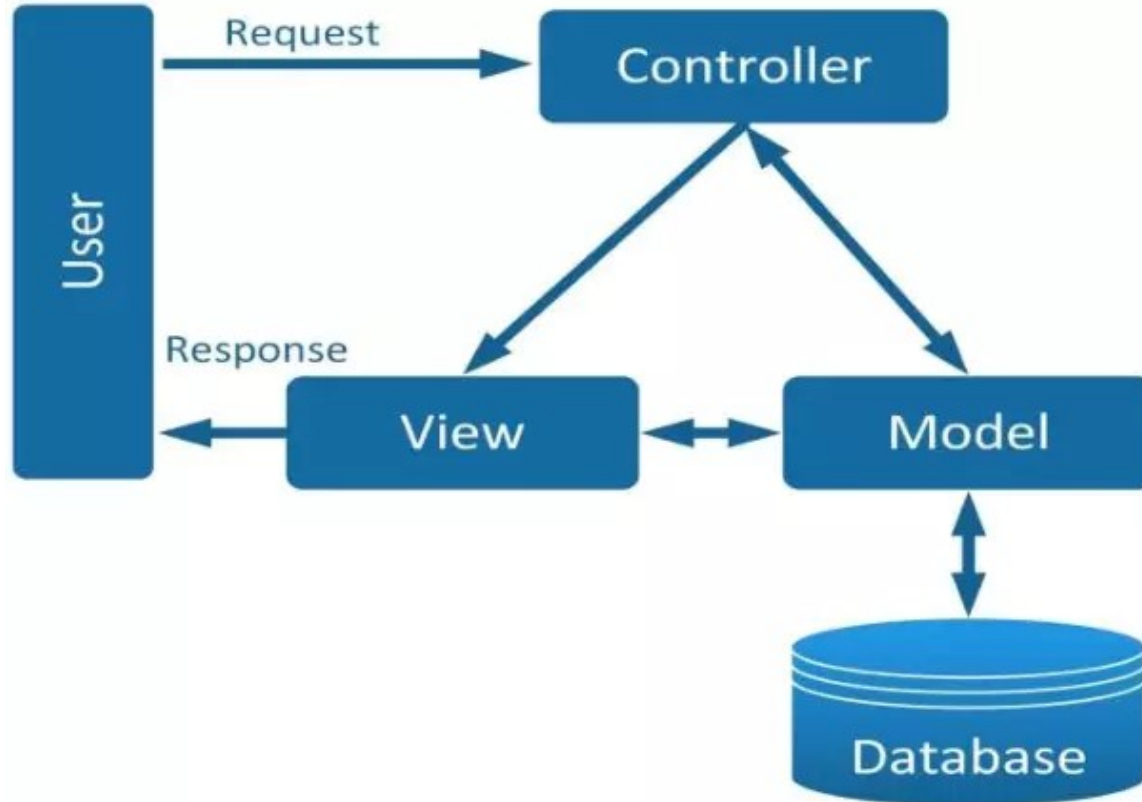

Application Development

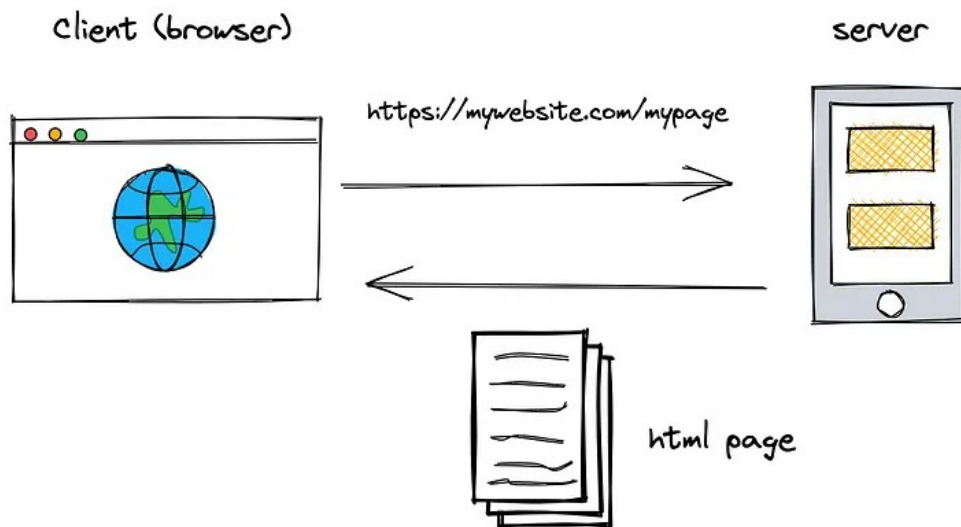


MVC Architecture

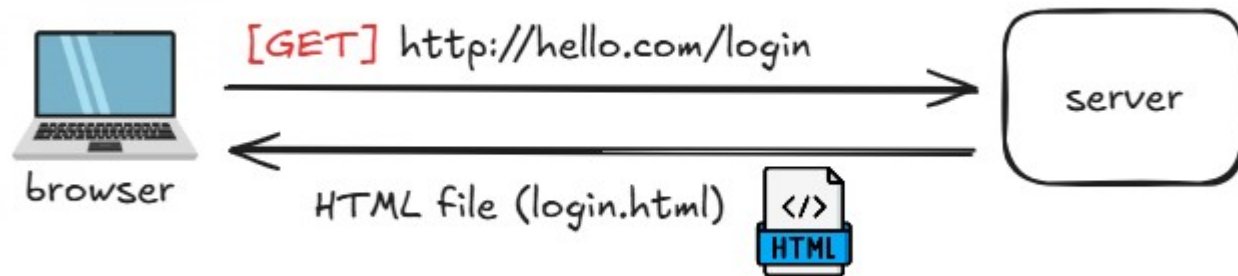


Servlet

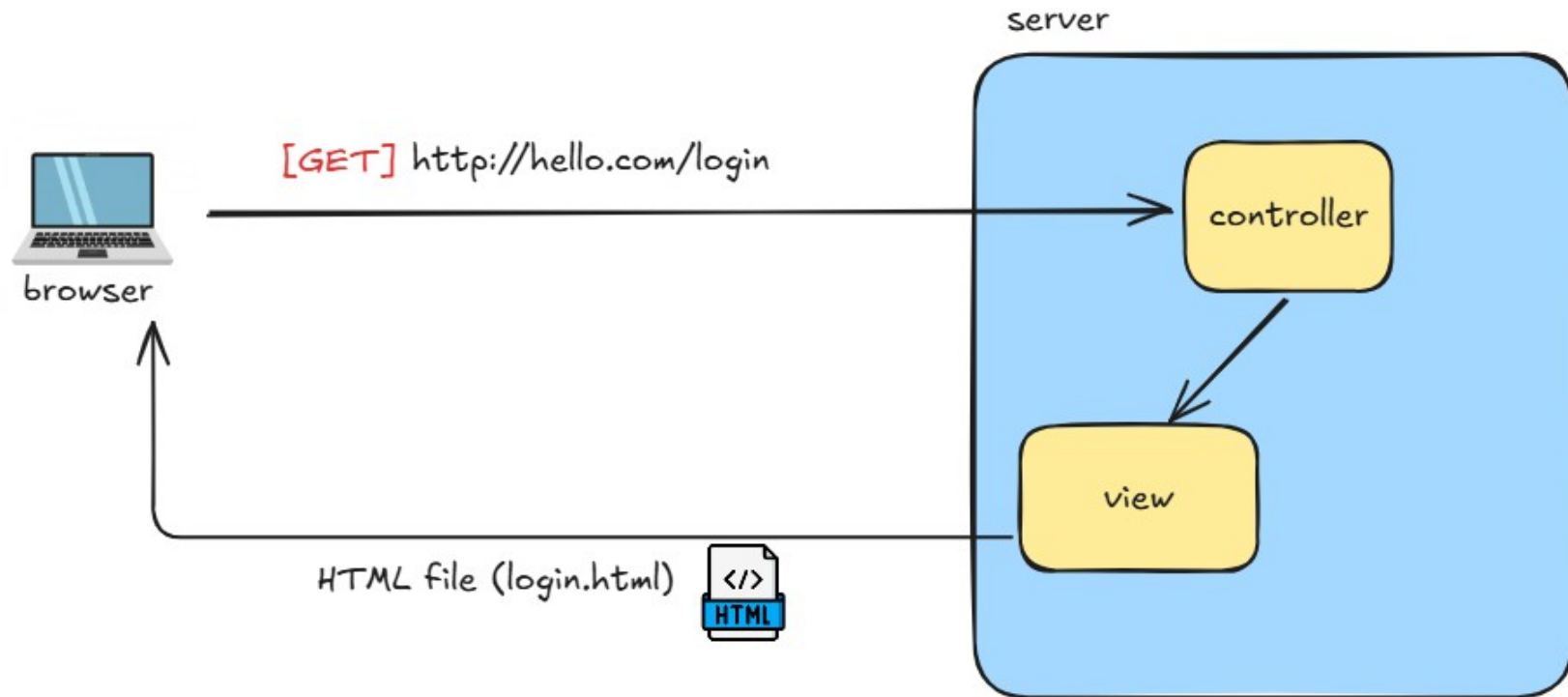
- ❖ Công nghệ server-side trong Java để xây dựng ứng dụng web
- ❖ Nhận và xử lý request từ client, trả về response



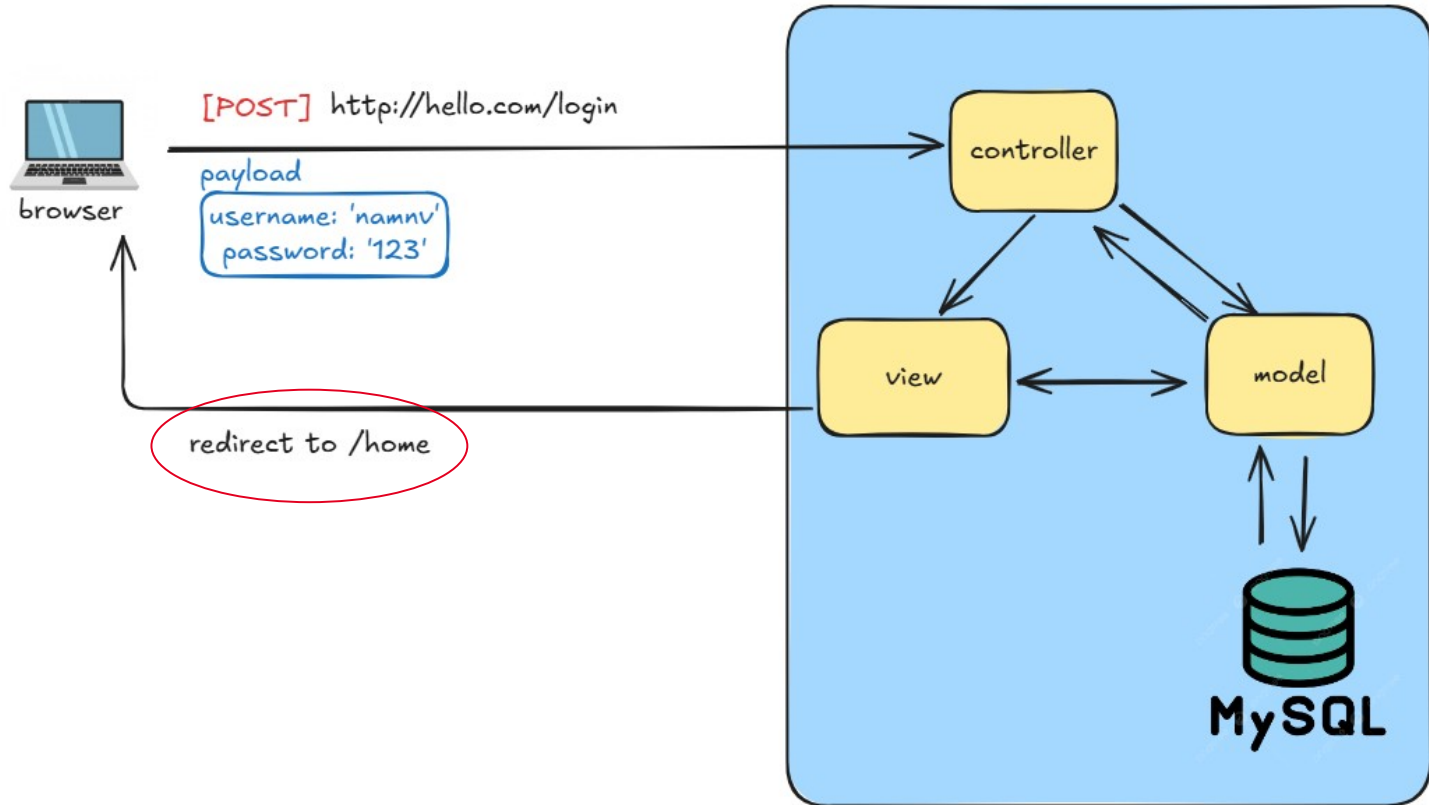
[Get] /login



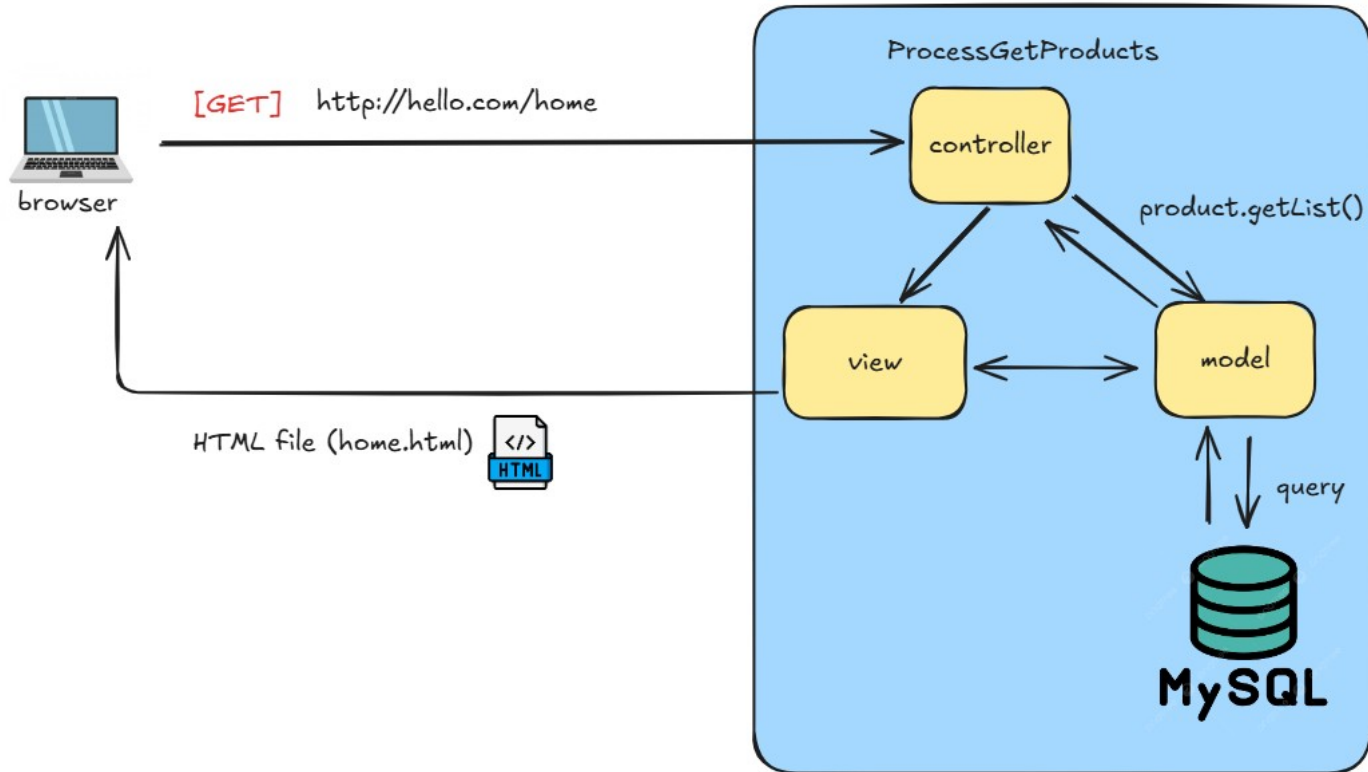
[Get] /login



[POST] /login

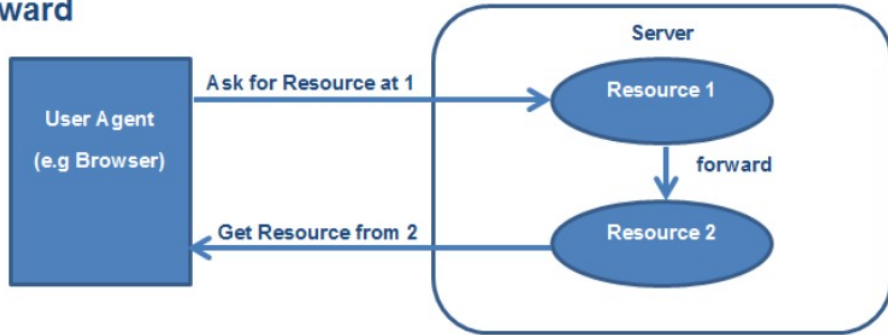


Redirect to [GET] /home

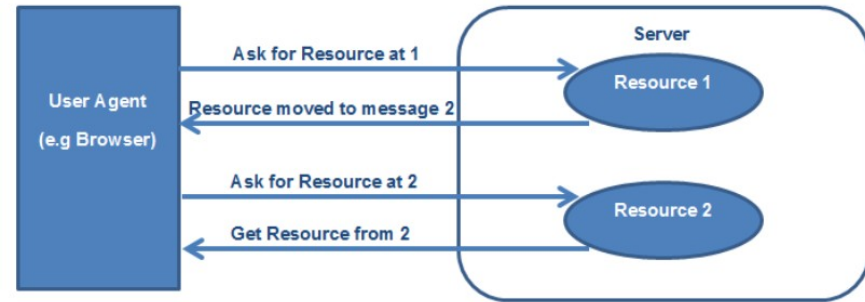


Forward VS Redirect

Forward



SendRedirect



Forward VS Redirect

Tiêu chí	Forward	Redirect
Trường hợp sử dụng	Chuyển xử lý trong cùng ứng dụng	Chuyển hướng đến URL khác . Người dùng thấy URL mới trong trình duyệt
URL hiển thị	Không thay đổi URL trên trình duyệt	Thay đổi URL trên trình duyệt
Số lượng request	Một request duy nhất	Hai request riêng biệt
Dữ liệu request	Giữ nguyên dữ liệu request	Mất dữ liệu request gốc

```
RequestDispatcher dispatcher =  
request.getRequestDispatcher("/target-servlet");  
dispatcher.forward(request, response);
```

```
httpResponse.sendRedirect(loginURI);
```

JDBC - Truy vấn (SELECT)

```
String sql = "SELECT * FROM users WHERE email = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "example@gmail.com");
ResultSet rs = pstmt.executeQuery();

while (rs.next()) {
    System.out.println(rs.getInt("id") + ", " +
        rs.getString("name") + ", " + rs.getString("email"));
}
```

JDBC - Thêm mới (INSERT)

```
String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "Nguyen Van A");
pstmt.setString(2, "vana@example.com");

int affectedRows = pstmt.executeUpdate();
System.out.println("Rows inserted: " + affectedRows);
```

JDBC - Cập nhật (UPDATE)

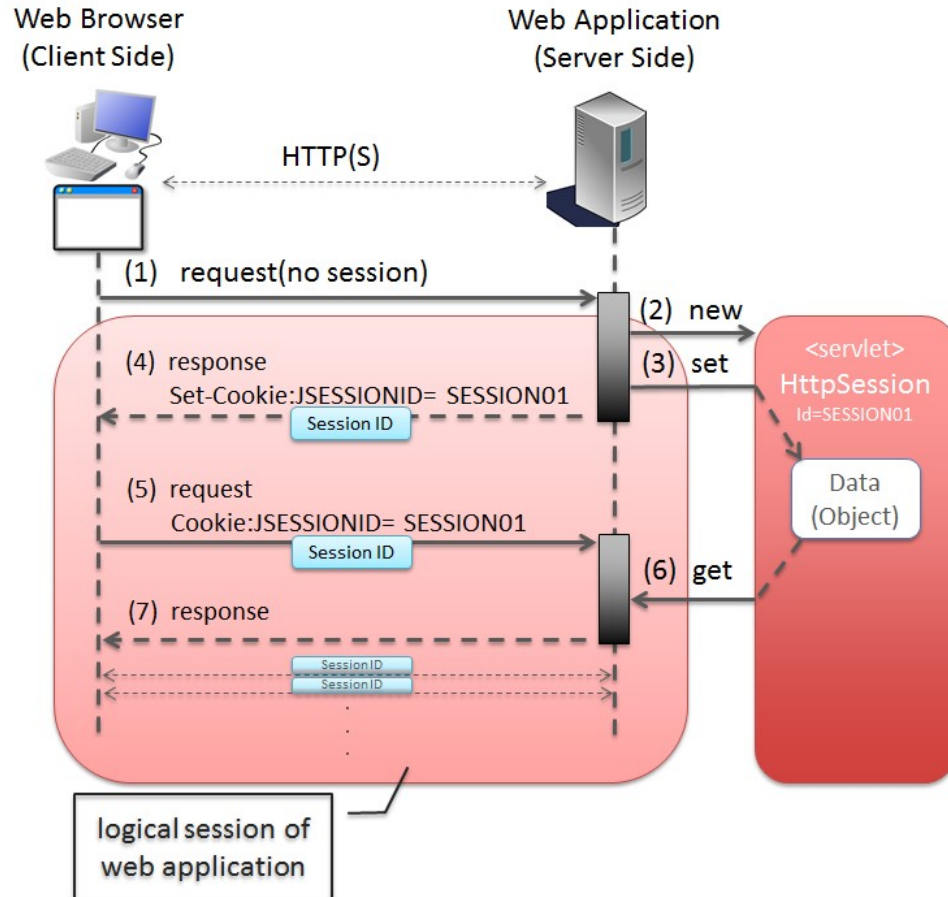
```
String sql = "UPDATE users SET name = ? WHERE email = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "Nguyen Van B");
pstmt.setString(2, "vana@example.com");

int affectedRows = pstmt.executeUpdate();
System.out.println("Rows updated: " + affectedRows);
```

HTTP

- ❖ HTTP (Hypertext Transfer Protocol) là một giao thức stateless
 - Mỗi request HTTP được xử lý hoàn toàn độc lập, không phụ thuộc vào các request trước đó
 - Không có mối liên hệ tự nhiên giữa hai request liên tiếp, ngay cả khi chúng đến từ cùng một người dùng

Session



Session

- ❖ Khi người dùng truy cập website lần đầu, server tạo một **session ID** duy nhất
- ❖ Session ID thường được lưu trong cookie của trình duyệt
- ❖ Server lưu trữ dữ liệu session tương ứng với session ID
- ❖ Mỗi request tiếp theo sẽ gửi kèm cookie chứa **session ID**
- ❖ Server dùng session ID để **xác định người dùng và truy xuất dữ liệu session**
- ❖ Thay đổi trình duyệt, thay đổi thiết bị (IP), sang ẩn danh => Session mới
 - session.setAttribute(name, value)
 - session.getAttribute(name)
 - session.removeAttribute(name)
 - session.invalidate()

Task 1

1. Tạo trang home tương ứng với /home.
2. Đăng nhập, lưu lại thông tin user vào session, chuyển hướng vào /home
3. Sang tab khác xem tính chất của session
4. Khi chưa đăng nhập thì không cho vào home
5. Khi đã đăng nhập mà vào /login thì sẽ redirect về /home
6. Sửa lại trang home thành hiển thị danh sách sản phẩm

Task 2

1. Trong trang danh sách sản phẩm, thêm search, sắp xếp, phân trang
2. Tạo trang chi tiết sản phẩm
3. Create / Update sản phẩm, đẩy ảnh lên cloudinary

Task 3

1. Sử dụng AJAX để nhắn tin
2. Sử dụng redis để hạn chế đăng nhập nhiều session
3. Sử dụng elastic search để tìm sản phẩm